计算机研究与发展
Journal of Computer Research and Development
DOI:10.7544/issn1000-1239.2020.20200198
57(9):1810-1822,2020

多层次算力网络中代价感知任务调度算法

刘泽宁[1,4,5] 李 凯[2,4] 吴连涛[2,4] 王 智[3,7] 杨 旸[1,2,4,6]
[1](上海科技大学信息科学与技术学院 上海 201210)
[2](上海科技大学创意与艺术学院 上海 201210)
[3](浙江大学控制科学与工程学院 杭州 310027)
[4](上海雾计算实验室(上海科技大学) 上海 201210)
[5](紫金山实验室 南京 211111)
[6](鹏城实验室网络通信研究中心 广东深圳 518000)
[7](工业控制技术国家重点实验室(浙江大学) 杭州 310027)
(liuzy1@shanghaitech.edu.cn)

CATS: Cost Aware Task Scheduling in Multi-Tier Computing Networks

Liu Zening[1,4,5], Li Kai[2,4], Wu Liantao[2,4], Wang Zhi[3,7], and Yang Yang[1,2,4,6]
[1](*School of Information Science and Technology, ShanghaiTech University, Shanghai* 201210)
[2](*School of Creativity and Art, ShanghaiTech University, Shanghai* 201210)
[3](*Department of Control Science and Engineering, Zhejiang University, Hangzhou* 310027)
[4](*Shanghai Institute of Fog Computing Technology (ShanghaiTech University), Shanghai* 201210)
[5](*Purple Mountain Laboratories, Nanjing* 211111)
[6](*Research Center for Network Communication, Peng Cheng Laboratory, Shenzhen, Guangdong* 518000)
[7](*State Key Laboratory of Industrial Control Technology (Zhejiang University), Hangzhou* 310027)

Abstract Due to more data and more powerful computing power and algorithms, IoT (Internet of things) applications are becoming increasingly intelligent, which are shifting from simple data sensing, collection, and representation tasks towards complex information extraction and analysis. The continuing trend requires multi-tier computing resources and networks. Multi-tier computing networks involve collaborations between cloud computing, fog computing, edge computing, and sea computing technologies, which have been developed for regional, local, and device levels, respectively. However, due to different features of computing technologies and diverse requirements of tasks, how to effectively schedule tasks is a key challenge in multi-tier computing networks. Besides, how to motivate multi-tier computing resources is also a key problem, which is the premise of the formation of multi-tier computing networks. To solve these challenges, in this paper, we propose a multi-tier computing network and a computation offloading system with hybrid cloud and fog, define a weighted cost function consisting of delay, energy, and payment, and formulate a cost aware task scheduling (CATS) problem. Furthermore, we propose a computation load based payment model to motivate cloud and fog, and include the payment related cost into the overall cost. To be specific, based on different features and requirements of cloud and fog, we propose a static payment

收稿日期:2020-03-20;**修回日期**:2020-06-08
基金项目:国家重点研发计划项目(2019YFB1803304);国家自然科学基金重点项目(61932014);华为项目(YBN2019125163)
This work was supported by the National Key Research and Development Program of China (2019YFB1803304), the Key Program of the National Natural Science Foundation of China (61932014), and the Huawei Technologies (YBN2019125163).
通信作者:杨旸(yangyang@shanghaitech.edu.cn)

model and a dynamic payment model for cloud and fog, respectively, which constitute the hybrid payment model. To solve CATS problem, we propose a potential game based analytic framework and develop a distributed task scheduling algorithm called CATS algorithm. Numerical simulation results show that CATS algorithm can offer the near-optimal performance in system average cost, and achieve more number of beneficial UEs (user equipment), compared with the centralized optimal method. Besides, it shows that the dynamic payment model may help fog obtain more income, compared with the static payment model.

Key words multi-tier computing network; fog computing; edge computing; computation offloading; task scheduling; incentive mechanism; potential game

摘　要 随着越来越多数据的产生以及更加强大的算力和算法的运用,物联网应用也变得越来越智能.典型的物联网应用也从简单的数据感知、收集和表示转向复杂的信息提取和分析.这一持续的趋势需要多层次算力资源及网络.多层次算力网络涉及云计算、雾计算、边缘计算和海计算等技术之间的相互协作,分别针对区域级别、本地级别和设备级别的物联网应用.但是,由于计算技术的不同特征以及任务的不同需求,如何有效地进行任务调度是多层次算力网络中的一个关键挑战.此外,如何激发多层次算力资源的积极性也是一个关键问题,这是多层次算力网络得以成形的前提.为解决上述挑战,提出了一个云雾混合多层次算力网络及计算卸载系统,定义了一个由时延、能耗及付费组成的加权代价函数,并建模了一个代价感知任务调度(cost aware task scheduling,CATS)问题.而且,为激发云和雾的积极性,提出了一个基于计算量的付费模型并将付费相关代价也考虑进总代价.具体来说,根据云和雾的不同特性和需求,分别提出了一个静态付费模型和动态付费模型,从而构建了一个混合付费模型.为解决上述CATS问题,提出了一个基于势博弈的分析框架,并设计了一个分布式任务调度算法——CATS算法.数值仿真结果表明,与集中式最优方法相比,CATS算法可以在系统平均代价方面提供近似最优的性能,并让更多用户受益.此外,与静态付费模型相比,动态付费模型可能可以帮助雾获得更多收入.

关键词 多层次算力网络;雾计算;边缘计算;计算卸载;任务调度;激励机制;势博弈

中图法分类号 TP393

随着越来越多数据的产生以及更加强大的算力算法的运用,物联网应用也变得越来越智能.典型的物联网应用也从简单的数据感知、收集和表示转向复杂的信息提取和分析.未来,物联网可广泛应用于环境监测、城市管理、医疗健康等任务.这些任务往往需要实时的数据处理、信息提取和分析决策.但是,由于有限的通信带宽、不稳定的网络连接以及严格的时延要求等,仅仅依靠云计算已无法支持无处不在且日渐强大的物联网部署和应用.为应对这一挑战,多层次算力网络[1]被提了出来.

多层次算力网络涉及云计算、雾计算[2]、边缘计算[3]和海计算[4]等技术之间的相互协作.这些技术的特点和覆盖范围不同,可以分别针对区域级别、本地级别和设备级别的物联网应用和服务.云计算位于最上层,远离终端设备.其通常是集中式的,用于全局任务.边缘计算靠近终端设备,其通常是分布式的,擅长处理本地的实时任务.雾计算位于云计算和边缘计算中间,起到桥梁的作用,是管理多层资源的关键.终端设备在设备层面上形成了海计算的概念,以支持实时的数据感知、控制执行等一些基本功能.因此,这样一个多层次算力网络可以显著提升终端设备的服务质量和用户体验,节省时延和能耗.

但是,在这样一个多层次算力网络下,如何有效地进行任务调度是个挑战.首先,云、雾、边缘等计算技术特点各异、侧重不同,如上所述.其次,不同终端设备、用户、任务需求各异.有的对时延敏感,有的则更在意能耗.如何根据设备/用户/任务的不同需求,将任务调度到不同特点和网络层次的算力资源上,是一个巨大的挑战.这也是实现多层次算力网络功效的一个重要方面.另外,如何考虑或设计激励机制以激励网络各层次资源参与多层次算力网络也是我们需要思考的问题,因为这是多层次算力网络得以成形的前提.

为解决上述挑战,本文提出了一个云雾混合

多层次算力网络及计算卸载系统，定义了一个由时延、能耗及付费组成的加权代价函数，并建模了一个代价感知任务调度(cost aware task scheduling，CATS)问题.而且，为激发云和雾的积极性，我们提出了一个基于计算量的付费模型并将付费相关代价也考虑进总代价.具体来说，根据云和雾的不同特性和需求，我们分别提出了一个静态付费模型和动态付费模型，从而构建了一个混合付费模型.为解决上述CATS问题，我们提出了一个基于势博弈的分析框架，并设计了一个分布式任务调度算法——CATS算法.

本文的主要贡献包括3个方面：

1) 提出了一个云雾混合多层次算力网络及计算卸载系统，定义了一个由时延、能耗及付费组成的加权代价函数，并建模了一个代价感知任务调度问题——CATS问题.

2) 提出了一个基于计算量的付费模型以激励云和雾参与到多层次算力网络及服务架构，并进一步根据云和雾的不同特性和需求分别设计了一个静态付费模型和动态付费模型，从而构建了一个混合付费模型.数值仿真结果表明，和静态付费模型相比，动态付费模型可以帮助雾服务提供商在用户数较大时获得更多收入.

3) 提出了一个基于势博弈的问题分析框架，并设计了一个分布式任务调度算法——CATS算法.数值仿真结果表明，和集中式最优方法相比，CATS算法既可以取得近似最优的系统平均代价，又可以让更多用户从任务卸载中受益，即CATS算法在系统整体及用户个体2个维度上均具有优秀表现.

1 相关工作

关于任务调度或计算卸载的研究可追溯到传统的云计算.文献[5-6]针对云计算，给出了一个一般的计算卸载策略.

随着雾/边缘计算等技术的出现，有关雾/边缘计算中的任务调度或计算卸载的研究涌现，但大多数都是针对单层或两层算力网络架构，即物-物、雾-雾或物-雾/边缘，而忽略了云的存在及作用[7-13].文献[7-8]分别针对多个用户单个边缘节点的网络场景和多个用户多个边缘节点的网络场景，研究了以最小化时延为目标的任务调度问题，并分别推导了闭式最优解，提出了一个分布式任务调度算法.文献[9]提出了一个新颖的基于D2D(device to device)的计算卸载框架，其中设备之间可以互相进行计算卸载.文献[9]采用李雅普诺夫(Lyapunov)方法设计了一个在线任务调度算法以优化系统平均能耗.文献[10-11]针对多个用户单个边缘节点的网络场景，研究了以最小化时延及能耗代价为目标的任务调度问题，并提出了一个统一的基于博弈论的分布式任务调度算法.和文献[10-11]不同的是，文献[12-13]针对多个用户多个边缘节点的网络场景，研究了以最小化时延及能耗代价为目标的任务调度问题.它们分别利用博弈论的方法设计了一个分布式任务调度算法.另外，文献[13]还考虑了通信资源及计算资源的分配问题.

最近，多层次算力网络中的任务调度或计算卸载的研究愈发得到人们关注[14-18].文献[14-15]研究了以最小化时延为目标的任务调度问题，并分别提出了一个基于整数规划的集中式任务调度算法和一个基于匹配理论的分布式任务调度算法.文献[16]研究了以最小化功耗为目标的任务调度及资源分配问题.文献[16]将问题分解为3个子问题，并分别通过内点法、广义弯曲算法及匈牙利算法求解.和文献[9]类似，文献[17]提出了一个新颖的基于D2D的混合计算卸载架构，并设计了一个基于三层图匹配的任务调度算法以解决该卸载框架下以最小化时延及能耗代价为目标的任务调度问题.文献[18]则针对多用户的物联网场景，研究了以最小化时延及能耗代价为目标的任务调度问题，并利用博弈论设计了一个分布式方法.以上工作都没有考虑雾/边缘节点及云的激励问题.

另一方面，一些工作也关注到了计算卸载中的激励机制问题[9,19-23].对于单层算力网络而言，如果同一网络层节点之间可以相互进行计算卸载，那么我们一般可以采用“tit-for-tat”的激励机制，即贡献资源多的个体相应也可以享受更多的资源[9,19].文献[9,19]分别提出了一个基于D2D和同构雾计算网络的计算卸载框架.在文献[9]和文献[19]中，设备(或雾节点)与设备(或雾节点)之间可以相互共享通信资源及计算资源，从而相互进行计算任务的卸载.这样一个可以相互共享资源的单层算力网络类似于点对点(peer-to-peer)系统，因此一般可以采用“tit-for-tat”的激励机制以激励个体参与资源共享.如果是跨层或多层的算力网络架构，那么不同网络层或实体之间可以通过付费的方式共享资源[20-23].文献[20]中，用户需向边缘节点付费以使用其计算

资源.文献[21]中,网络运营商可以租用雾节点服务终端用户,并根据雾节点提供的计算资源及服务质量付费.文献[22]中,云服务运营商可以通过付费的方式将计算卸载到边缘服务器上.文献[23]中,用户可以通过付费的方式向数据服务运营商订阅服务,而数据服务运营商则可以通过付费的方式来利用边缘雾节点的资源更好地服务终端用户.

综上所述,国内外的研究人员针对多层次算力网络中的任务调度或计算卸载问题已经做了一些研究和探索,但其中很少有同时考虑云和雾或者边缘的激励问题的.大多数研究工作要么只集中于计算卸载过程中的任务调度或资源分配,要么主要研究激励问题,且网络场景多为单层或两层算力网络.与此同时,相较分布式算法,大多数工作在设计任务调度或者计算卸载算法时采用的是集中式算法.鉴于以上考虑,本文将提出一个云雾混合多层次算力网络及计算卸载系统,定义一个由时延、能耗及付费组成的加权代价函数,建模一个代价感知任务调度(CATS)问题并相应地提出一个分布式任务调度算法.而且,为激发云和雾的积极性,我们将提出一个基于计算量的付费模型并将付费相关代价也考虑进总代价.具体来说,根据云和雾的不同特性和需求,我们将分别提出一个静态付费模型和动态付费模型,从而构建一个混合付费模型.

2 多层次算力网络

在本节中,我们主要介绍多层次算力网络的相关概念及特性.

如图1所示,多层次算力网络涉及云计算、雾计算、边缘计算和海计算等技术之间的相互协作.这些技术的特点和覆盖范围不同,可以分别针对区域级别、本地级别和设备级别的物联网任务和应用.云计算位于最上层,远离终端设备,其通常是集中式的,用于全局任务.边缘计算靠近终端设备,其通常是分布式的,擅长处理本地的实时任务.雾计算位于云计算和边缘计算中间,起到桥梁的作用,是管理多层资源的关键.终端设备在设备层面上形成了海计算的概念,以支持实时的数据感知、控制执行等一些基本功能.

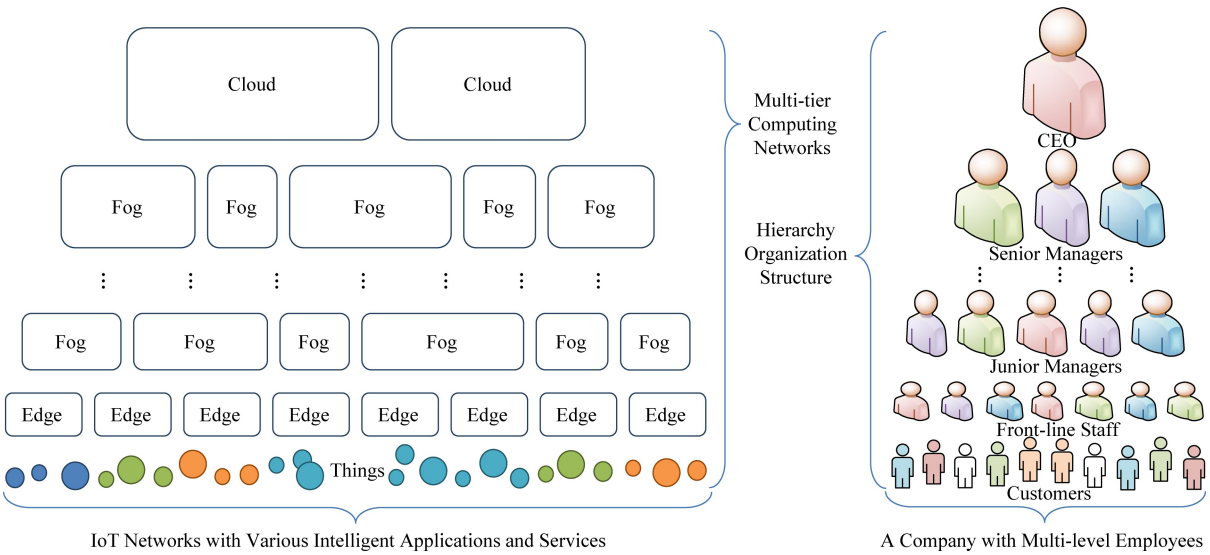

Fig. 1 A multi-tier computing network architecture which integrates cloud, fog, edge and sea computing technologies[1]

图1 一个综合了云、雾、边缘和海计算技术的多层次算力网络架构[1]

这样一个多层次算力网络可以抽象为一个自上而下,具有多层组织的大公司结构,如图1所示.在一个公司中,各级管理者和员工通常拥有不同的资源和权限来访问、处理、分配任务,发展客户以及做出决策.云计算相当于公司的最高层:拥有最多的信息源、最强的分析能力、最大的存储空间以及最高的决策权.因此,云计算负责处理具有挑战性的全局任务,如跨域的数据分析与处理、异常行为的诊断与跟踪、隐藏问题的预测与搜索、新知识的发现与创造、长期战略的规划与决策等.另一方面,边缘计算相当于一线员工,他们拥有的资源和权限最少,但可以与不同行业的客户直接打交道.因此,边缘计算擅长处理本地级别的时延敏感型任务,例如数据收集、数据压缩、信息提取和事件监视.雾计算位于云计算和

边缘计算中间,相当于公司的中层管理人员.和一个具有多层资源和职责的高效管理体系一样,雾计算是多层次的结构,每一层可以共享计算、通信、存储等资源.由于用户需求在时间上和空间上是动态变化的,因此雾计算可以提供一种灵活的方法,将网络中分散在不同地理位置或逻辑位置的资源整合在一起,从而为任何客户提供及时有效的服务.物联网网络中的设备或"物"相当于公司的客户,它们对智能应用和服务有着不同的需求.尽管每个设备的处理、通信、存储和电力资源有限,但它们在设备层面上形成了海计算的概念,以支持实时的数据感知、环境认知、移动控制和其他一些基本功能.

一个公司的成功离不开不同职级和部门之间有效的沟通和协作机制.同样地,云计算、雾计算、边缘计算和海计算之间的交互和协作对于构建多层次算力网络架构至关重要.因此,该架构应连接网络中所有节点共享的计算、通信和存储资源,充分利用它们在不同位置和层次的能力,并根据用户的动态需求提供智能、及时、高效的服务.因此,这种多层次算力网络可以显著提高服务质量和用户体验,同时节省时间、资源和成本.

但是应该指出的是,多层次算力网络的发展和标准化才刚刚开始,还有许多技术挑战有待解决.具体来说,在安全、信任和隐私方面,终端用户应该在不牺牲个人数据的前提下,使用附近的共享计算节点.再者,需要开发资源虚拟化和资源可视化功能,以便在不同的机器和硬件上共享物理计算资源.此外,高效的业务流程,即异构资源和服务的管理及分配,需要网络节点之间的互相操作.最后,为服务多组用户,即满足多租户需求,需要开发一种新的商业模式,并构建生态系统以鼓励共享资源和协作服务.这些问题的解决需要跨学科的研究和合作[1].

3 基于多层次算力网络的计算卸载系统

3.1 系统概述

本节将详细介绍一个云雾混合多层次算力网络及计算卸载系统,定义一个由时延、能耗及付费组成的加权代价函数,并建模一个代价感知任务调度(CATS)问题.

如图2所示,该云雾混合多层次算力网络由多个边缘雾节点(fog node, FN)和一个远端的云组成,它们共同向终端用户(user equipment,UE)提供云计算类服务.其中,云由云服务提供商运营,如亚马逊、微软、阿里巴巴,而边缘雾节点由雾服务提供商管理,如电信运营商(电信运营商可以通过在已有基站或接入点附近安装服务器的方式为用户提供存储、计算等云计算服务).在本文中,我们假设云由单个云服务提供商运营,而边缘雾节点也由单个雾服务提供商管理.用户可以通过付费的方式获取云服务提供商或雾服务提供商所提供的服务.

用户出于时延或能耗的考虑,可以将其计算任务卸载到雾节点或云上进行处理,这取决于二者提供的服务质量,即任务卸载过程中的时延或/和能耗,以及用户为此支付的费用.一般来说,云拥有强大的计算能力,但远离终端用户,通过广域网为终端用户提供服务,通信时延较大;而雾节点靠近终端用户,可以通过局域网为用户提供服务,通信时延较小,但单个雾节点计算能力有限.用户如何根据其自身不同的时延、能耗以及付费的考虑从而决定任务的处理位置是本文考虑的重点.

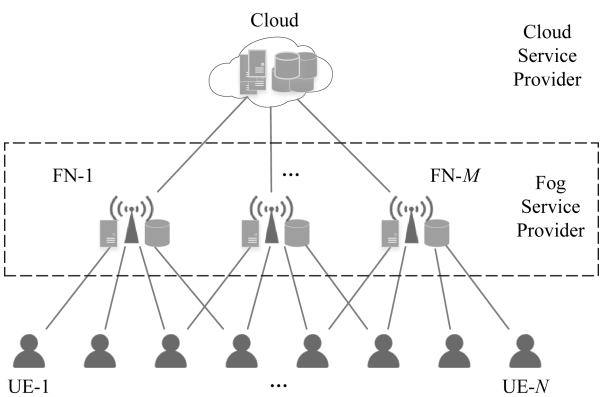

Fig. 2 An illustration of multi-tier computing network and computation offloading system with hybrid cloud and fog

图2 云雾混合多层次算力网络及计算卸载系统

为便于表达和分析,我们用 $\mathcal{N}=\{1,2,\cdots,N\}$,$\mathcal{M}=\{1,2,\cdots,M\}$ 和 c 来分别表示用户的集合、雾节点的集合以及云.另外,我们将可服务于用户 $n\in\mathcal{N}$ 的雾节点的集合记作 $\mathcal{M}_n$,其可由用户和雾节点之间实际的通信链路所决定.我们用一个参数二元组 (z_n,γ_n) 来表示用户 n 的计算任务,其中 z_n 是任务的大小(位),γ_n 是任务的处理密度[8,17-18](CPU 转数/位),即处理单个位的数据所需要的 CPU 转数.为方便起见,我们有时将用户 n 的任务称作任务 n.用户 n 可以选择在本地处理其计算任务,或者将其计算任务卸载至某个可服务的雾节点或云.我们将用户 n 的卸载策略记作 a_n,$a_n\in\mathcal{A}_n$,$\mathcal{A}_n=\{0,\mathcal{M}_n,c\}$,其中 0 代表本地计算,并且我们将 $\mathcal{A}_n$ 称作用户 n 的卸载策略空间.我们将所有用户的卸载策略称作卸载策略配置,并记作 $\boldsymbol{a}=(a_1,a_2,\cdots,a_N)$.给定策略配置 $\boldsymbol{a}$,我们将所有选择雾节点 m 的用户数记作 $n_m(\boldsymbol{a})$,$n_m(\boldsymbol{a})=\sum_{n=1}^{N}I_{\{a_n=m\}}$,其中 $I_{\{x\}}$ 是指示函数.如果 x 为真,则 $I_{\{x\}}=1$;否则,$I_{\{x\}}=0$.

接下来,我们将分别介绍该多层次算力网络及计算卸载系统下的通信模型、计算模型及付费模型,及其相关的时延、能耗及付费代价.

3.2 通信模型

由于雾节点靠近终端用户,因此用户可以通过局域网接入雾节点.我们用 $R_{n,m}$ 表示,当用户 n 独占雾节点 m 的信道时,从用户 n 到雾节点 m 的传输速率,其取决于物理层信号特征、发射功率、信道增益等.当多个用户同时将计算任务卸载到雾节点 m 上时,因为单个雾节点的通信资源有限,所以不同用户需要共享或竞争雾节点的通信资源.根据带宽共享模型[24-25],当多个用户将计算任务卸载到雾节点 m 上时,用户 n 到雾节点 m 的传输速率可以建模为

$$R_{n,m}(\boldsymbol{a})=\frac{R_{n,m}}{n_m(\boldsymbol{a})}. \tag{1}$$

该模型广泛运用于基于 TDMA 和 OFDMA 的 MAC 协议中带宽共享的建模.因此,用户 n 将任务卸载到雾节点 m 上的通信时延可表示为

$$T_{n,m}^{\text{trans}}=\frac{n_m(\boldsymbol{a})z_n}{R_{n,m}}. \tag{2}$$

这里,和大多数工作[7-9,12,18]一样,我们忽略结果返回至用户的过程,因为和任务本身大小相比,结果可以忽略不计.

相应地,用户 n 将任务卸载到雾节点 m 上的能耗可表示为

$$E_{n,m}^{\text{trans}}=p_n^{\text{fog}}\frac{n_m(\boldsymbol{a})z_n}{R_{n,m}}, \tag{3}$$

其中,p_n^{fog} 是用户 n 与雾节点通信的发射功率.这里我们假设用户在发送数据时功率恒定,因此功率控制并不在本文内容的考虑范围之内.

和雾节点不同的是,云远离终端用户,一般通过广域网与用户连接,其通信时延包括与接入点之间的传输时延以及任务从接入点往返云服务器的往返时延.我们假设每个用户 n 都可以以给定的传输速率 $R_{n,c}$ 接入网络,并用 t_n^{rt} 表示任务 n 的往返时延.因此,如果用户 n 选择将任务卸载到云上,那么其通信时延及能耗可分别表示为

$$T_{n,c}^{\text{trans}}=\frac{z_n}{R_{n,c}}+t_n^{\text{rt}}, \tag{4}$$

$$E_{n,c}^{\text{trans}}=p_n^{\text{cloud}}\frac{z_n}{R_{n,c}}, \tag{5}$$

其中,p_n^{cloud} 是用户 n 与云通信的发射功率.

3.3 计算模型

本地计算的时延可以表示为

$$T_{n,l}^{\text{comp}}=\frac{z_n\gamma_n}{f_n}, \tag{6}$$

其中,f_n 是用户 n 的本地计算能力,即 CPU 的时钟频率(CPU 转数/秒).

本地计算的能耗可以表示为[9-10,12,14,17-18]

$$E_{n,l}^{\text{comp}}=\kappa_n z_n\gamma_n f_n^2, \tag{7}$$

其中,κ_n 是与硬件结构相关的常数.

同样地,当用户将任务卸载到雾节点或云上时,雾节点或云的计算时延也可以表示成类似的形式.但因为单个雾节点的计算能力有限,所以当多个用户将计算任务卸载到雾节点上时,不同用户需要共享或竞争雾节点的计算能力.通常情况下,分配给单个用户或任务的计算能力是用户数的非增函数.这里为了简单起见,我们假设雾节点的计算能力被均匀分配给其上的任务[24-25].因此,用户 n 将任务卸载到雾节点 m 上的计算时延可以表示为

$$T_{n,m}^{\text{comp}}=\frac{n_m(\boldsymbol{a})z_n\gamma_n}{f_m}, \tag{8}$$

其中,f_m 是雾节点 m 的计算能力.

而云因为计算能力通常充足,因此每个用户可以被分配以一定的云计算能力 f_c.因此,用户 n 将任务卸载到云上的计算时延可以表示为

$$T_{n,c}^{\text{comp}}=\frac{z_n\gamma_n}{f_c}. \tag{9}$$

3.4 付费模型

为补偿雾节点和云在处理用户计算任务过程中

的开销并激励它们参与到用户的任务卸载中来,雾节点和云会向用户收取一定的费用,因此用户会支付一笔费用给雾节点和云.这是最直接也最直观的激励方式[20-23].一般来说,任务的计算量越大,雾节点和云处理任务时的付出越多,如能耗,因此支付给雾节点和云的费用也应该越多.但是,对于雾节点而言,如果其服务的用户数越多,单个用户等待的时延及消耗的能耗越大.因此,用户支付给雾节点的费用也应该减少,以弥补增加的时延及能耗代价.从雾节点角度理解,雾节点为了吸引更多的用户使用自身的服务以增加潜在的收入,并且弥补因为服务用户数的增加而导致的用户服务质量的下降,雾节点会给予用户一定的价格优惠.

我们将支付给雾节点 m 的费用建模为

$$C_{n,m}=c^{\mathrm{fog}}(1-\alpha(n_m(\boldsymbol{a})-1))z_n\gamma_n, \tag{10}$$

其中,c^{fog}是一个常数,表示雾服务提供商对单位计算量的收费,为雾服务提供商对外统一报价;$\alpha\in(0,\alpha_{\max}]$是折扣率.我们将上述付费模型称作动态付费模型.

而支付给云的费用可相应建模为

$$C_{n,c}=c^{\mathrm{cloud}}z_n\gamma_n, \tag{11}$$

其中,c^{cloud}是一个常数,表示云服务提供商对单位计算量的收费.我们将上述付费模型称作静态付费模型.

不同的建模方式也可以从云和雾各自特点的角度予以理解、解释.云服务提供商一般都是大公司,且市场成熟,个体用户对其议价能力弱,因此其付费模式较为固定;而雾服务提供商则更多为个人或者小公司,属于新加入的市场玩家,用户对其议价能力较强,因此其付费模式可以动态变化.雾服务提供商为从云服务提供商手中抢夺市场份额,以及不同雾服务提供商之间的相互竞争,其愿意采取一定的降价策略以吸引更多用户,以增加最终收入.我们将在数值仿真部分来进一步分析这样一个混合付费模型.

3.5 问题建模

我们将用户的任务卸载代价建模成时延、能耗及付费的线性组合[9-12,16-17],即定义了一个加权代价函数.我们将时延、能耗及付费这些不同度量统一映射为同一量纲[21].

如果用户 n 选择在本地处理任务,其代价可以表示为

$$O_{n,l}=\lambda_n^{\mathrm{T}}\frac{z_n\gamma_n}{f_n}+\lambda_n^{\mathrm{E}}\kappa_n z_n\gamma_n f_n^2. \tag{12}$$

如果用户 n 选择将任务卸载到雾节点 m 上进行处理,其代价可以表示为

$$O_{n,m}=\lambda_n^{\mathrm{T}}\left(\frac{1}{R_{n,m}}+\frac{\gamma_n}{f_m}\right)z_n n_m(\boldsymbol{a})+\lambda_n^{\mathrm{E}}\frac{p_n^{\mathrm{fog}}}{R_{n,m}}z_n n_m(\boldsymbol{a})+\lambda_n^{\mathrm{C}}c^{\mathrm{fog}}(1-\alpha(n_m(\boldsymbol{a})-1))z_n\gamma_n. \tag{13}$$

如果用户 n 选择将任务卸载到云上进行处理,其代价表示为

$$O_{n,c}=\lambda_n^{\mathrm{T}}\left(\frac{z_n}{R_{n,c}}+t_n^{\mathrm{rt}}+\frac{z_n\gamma_n}{f_c}\right)+\lambda_n^{\mathrm{E}}p_n^{\mathrm{cloud}}\frac{z_n}{R_{n,c}}+\lambda_n^{\mathrm{C}}c^{\mathrm{cloud}}z_n\gamma_n. \tag{14}$$

式(12)~(14)中的 λ_n^{T},λ_n^{E} 和 λ_n^{C} 分别表示时延、能耗及付费的权重,并将这些不同度量映射为同一量纲.$0\leqslant\lambda_n^{\mathrm{T}},\lambda_n^{\mathrm{E}},\lambda_n^{\mathrm{C}}\leqslant 1$.对于时延敏感型用户,其 λ_n^{T} 相应更大;对于能耗敏感型用户,其 λ_n^{E} 相应更大;对于费用敏感型用户,其 λ_n^{C} 相应更大.

根据式(12)~(14),在策略组合 $\boldsymbol{a}$ 下,用户 n 的代价可以表示为

$$O_n(a_n,\boldsymbol{a}_{-n})=O_{n,l}I_{\{a_n=0\}}+\sum_{m=1}^{M}O_{n,m}I_{\{a_n=m\}}+O_{n,c}I_{\{a_n=c\}}, \tag{15}$$

其中 $\boldsymbol{a}_{-n}=(a_1,a_2,\cdots,a_{n-1},a_{n+1},\cdots,a_N)$,表示除了用户 n 之外其他所有用户的卸载策略,$\boldsymbol{a}=(a_n,\boldsymbol{a}_{-n})$.

在我们的问题中,每个用户需自主决定其卸载决策,以最小化自身代价.因此,我们可以将相应的任务调度问题建模为:

$$\min_{a_n}O_n(a_n,\boldsymbol{a}_{-n}). \tag{16}$$

我们将式(16)所述问题称作代价感知任务调度(CATS)问题.

因为用户之间的目标彼此矛盾,卸载决策相互耦合,所以 CATS 问题难以解决.从式(16)可知,用户的代价不仅取决于其自身的卸载策略,还取决于其他用户的卸载策略.为解决上述难题,接下来我们将把上述问题重新建模为一个非合作博弈,进而通过势博弈理论予以分析、解决.

4 基于势博弈的代价感知任务调度算法

在本节中,我们将详细介绍如何通过博弈论相关理论去高效解决 3.5 节的任务调度问题.

首先,我们将 CATS 问题重新建模为一个非合作博弈 G,其定义为 $G=(\mathcal{N},(\mathcal{A}_n)_{n\in\mathcal{N}},(O_n)_{n\in\mathcal{N}})$,其中 $\mathcal{N}$ 是玩家的集合,$\mathcal{A}_n$ 是玩家 n 的策略空间,

O_n 是玩家 n 的成本函数.我们将博弈 G 称为 CATS 博弈.

接下来,我们研究 CATS 博弈是否存在纯策略的纳什均衡解.在此之前,我们需要介绍 2 个定义.

定义 1. 给定 $\boldsymbol{a}_{-n}$,用户 n 的最佳响应函数 $b_n(\boldsymbol{a}_{-n})$是一组策略:

$$b_n(\boldsymbol{a}_{-n})=\{a_n \mid O_n(a_n,\boldsymbol{a}_{-n})\leqslant O_n(a_n',\boldsymbol{a}_{-n}),\forall a_n'\in\mathcal{A}_n\}. \tag{17}$$

定义 2. $\bar{\boldsymbol{a}}=(\bar{a}_1,\bar{a}_2,\cdots,\bar{a}_N)$是 CATS 博弈 G 的纯策略纳什均衡解,当且仅当

$$\bar{a}_n\in b_n(\bar{\boldsymbol{a}}_{-n}),\forall n\in\mathcal{N}. \tag{18}$$

纳什均衡解的解释如下:在纳什均衡解,每个玩家(在我们问题中为用户)可以在其他玩家策略(在我们问题中为卸载策略)不变的情况下,通过改变其自身策略而进一步减小其成本(在我们问题中为加权代价).

推论 1. CATS 博弈 G 拥有至少一个纯策略的纳什均衡解并具有有限提升性,如果

$$B_n\triangleq\lambda_n^{\mathrm{T}}\frac{z_n\gamma_n}{f_n}+\lambda_n^{\mathrm{E}}\kappa_n z_n\gamma_n f_n^2-\lambda_n^{\mathrm{C}}c^{\mathrm{fog}}(1+\alpha)z_n\gamma_n>0,\quad\forall n\in\mathcal{N}, \tag{19}$$

$$D_{n,m}\triangleq\left[\lambda_n^{\mathrm{T}}\left(\frac{1}{R_{n,m}}+\frac{\gamma_n}{f_m}\right)+\lambda_n^{\mathrm{E}}\frac{p_n^{\mathrm{fog}}}{R_{n,m}}-\lambda_n^{\mathrm{C}}c^{\mathrm{fog}}\alpha\gamma_n\right]z_n>0,n\in\mathcal{N},m\in\mathcal{M}, \tag{20}$$

$$F_n\triangleq\lambda_n^{\mathrm{T}}\left(\frac{z_n}{R_{n,c}}+t_n^{\mathrm{rt}}+\frac{z_n\gamma_n}{f_c}\right)+\lambda_n^{\mathrm{E}}p_n^{\mathrm{cloud}}\frac{z_n}{R_{n,c}}+\lambda_n^{\mathrm{C}}c^{\mathrm{cloud}}z_n\gamma_n-\lambda_n^{\mathrm{C}}c^{\mathrm{fog}}(1+\alpha)z_n\gamma_n>0,\forall n\in\mathcal{N}. \tag{21}$$

证明. 我们可以证明 CATS 博弈 G 是一个广义势博弈[27],该势博弈的势函数为

$$\Phi(\boldsymbol{a})=\sum_{m=1}^{M}\sum_{i=1}^{n_m(\boldsymbol{a})}\log(i)-\sum_{m=1}^{M}\sum_{n=1}^{N}\log(B_n)I_{\{a_n=m\}}-\sum_{n=1}^{N}\log(B_n)I_{\{a_n=c\}}+\sum_{m=1}^{M}\sum_{n=1}^{N}\log(D_{n,m})I_{\{a_n=m\}}-\sum_{m=1}^{M}\sum_{n=1}^{N}\log(F_n)I_{\{a_n=m\}}-\sum_{n=1}^{N}\log(F_n)I_{\{a_n=0\}}, \tag{22}$$

从而

$$O_n(a_n',\boldsymbol{a}_{-n})-O_n(a_n,\boldsymbol{a}_{-n})<0\Rightarrow\Phi(a_n',\boldsymbol{a}_{-n})-\Phi(a_n,\boldsymbol{a}_{-n})<0,\quad\forall a_n,a_n'\in\mathcal{A}_n,\boldsymbol{a}_{-n}\in\prod_{n'}\mathcal{A}_{n'}. \tag{23}$$

证毕.

具体证明过程可以参考我们补充的在线材料[26].每个广义势博弈都存在至少一个纯策略的纳什均衡解.而且,无论以什么为起点,每个较优或最佳响应序列都收敛于一个纯策略的纳什均衡解,即有限提升性[27].

式(20)中,$D_{n,m}$表示雾节点 m 每多服务一位用户,其所服务的用户 n 增加的计算卸载的代价.这一值在实际中一般为正数,否则雾节点可以服务越来越多的用户,而这显然与实际不符.B_n 和 F_n 的值一般也为正数,否则用户会因为费用项过大而选择本地计算,博弈则失去意义.在实际仿真时,我们可以通过限制用户的决策空间得已保证这些条件得到满足.

利用有限提升性,我们可以设计一个分布式算法——CATS 算法,去寻找 CATS 博弈的一个纯策略纳什均衡解.我们将时间划分为多个决策时隙[8,10-12],进而整个算法如算法 1 所示.

算法 1. 代价感知任务调度 CATS 算法.

输入:用户集合 $\mathcal{N}$、雾节点集合 $\mathcal{M}$、云 c 及其相关参数;

输出:用户卸载策略配置 $\boldsymbol{a}$.

① 初始化:
② 每个用户 n 选择本地计算,即 $a_n(0)=0$;
③ 云向用户广播其计算能力及付费模型等参数,即 f_c,c^{cloud};
④ 雾节点向用户广播其计算能力及付费模型等参数,即 f_m,c^{fog}及 α;
⑤ repeat 每个决策时隙,每个用户 n 及雾节点 m 并行执行以下操作:
⑥ 雾节点 m 向用户广播其负载 $n_m(\boldsymbol{a}(t))$;
⑦ 用户 n 计算其最佳响应函数 $b_n(\boldsymbol{a}_{-n}(t))$;
⑧ if $a_n(t)\notin b_n(\boldsymbol{a}_{-n}(t))$
⑨ if $0\in b_n(\boldsymbol{a}_{-n}(t)$或 $c\in b_n(\boldsymbol{a}_{-n}(t))$
⑩ 在下个决策时隙直接更新其任务卸载策略,即 $a_n(t+1)=0$ 或 $a_n(t+1)=c$;
⑪ else
⑫ 发送请求更新的信息至雾节点以竞争更新任务卸载策略的机会;
⑬ if 被允许更新任务卸载策略
⑭ 在下个决策时隙直接更新其任务卸载策略,即 $a_n(t+1)\in b_n(\boldsymbol{a}_{-n}(t))$;
⑮ else
⑯ 在下个决策时隙维持当前任务卸载策略,$a_n(t+1)=a_n(t)$;
⑰ end if
⑱ end if

⑲ else

⑳ 在下个决策时隙维持当前任务卸载策略，$a_n(t+1)=a_n(t)$；

㉑ end if

㉒ until 没有用户想更新其当前任务卸载策略.

我们进一步分析了上述分布式算法的收敛复杂度，如推论 2 所示.

推论 2. 如果 $B_n, D_{n,m}, F_n, \forall n\in\mathcal{N}, m\in\mathcal{M}$ 是整数，那么 CATS 算法至多经过 $N\dfrac{\log N-\log(H^{\min})}{\log(K^{\min})}$ 次决策时隙即可收敛，其中

$$H^{\min}=\min_{n,m}\left\{B_n, F_n, \frac{B_nF_n}{D_{n,m}}\right\}, \tag{24}$$

$$K^{\min}=\min_{n,m}\left\{\frac{1}{B_n}, \frac{1}{F_n}, \frac{1}{ND_{n,m}}\right\}+1. \tag{25}$$

具体证明过程可以参考我们补充的在线材料[26].

5 实验与结果

在本节中，我们通过数值仿真来评估 CATS 算法及动态付费模型的表现.

5.1 仿真设置

我们考虑一个由多个用户、10 个雾节点及云组成的多层次算力网络，其中每个用户可以使用 LTE 将计算任务卸载到遥远的云上，或 WiFi 将计算任务卸载到附近的雾节点上.为方便起见，我们假设每个用户都可以使用所有雾节点及云的服务.另外，我们假设每个计算任务的大小 z_n（单位是 KB）和处理密度 γ_n（单位是 cycle/b）分别从[500, 5 000]和[500, 3 000]中随机选取.

通信方面，LTE 和 WiFi 的平均传输速率为 5.85 Mb/s 和 3.01 Mb/s[18,28]，因此我们假设 LTE 的传输速率 $R_{n,c}$（单位是 Mb/s）和 WiFi 的传输速率 $R_{n,m}$（单位是 Mb/s）分别均匀分布于[4.85, 6.85]和[2.01, 4.01][17,28].云计算的往返时延 t_n^{rt} 设为 3 s.另外，我们将用户通过 LTE 通信的传输功率 p_n^{cloud} 和通过 WiFi 通信的传输功率 p_n^{fog} 分别设为 2 605 mJ/s[18,29] 和 1 224.78 mJ/s[18,28].

计算方面，我们假设每个用户的本地计算能力为 f_n，即本地设备的 CPU 时钟频率（单位是 GHz），从{0.8, 0.9, 1.0, 1.1, 1.2}中随机选取[7]，而云提供的计算能力 f_c 和雾节点的计算能力 f_m 分别设为 10 GHz 和 20 GHz.另外，本地计算的能耗常数 $\kappa_n=10^{-27}$[13,30].

付费方面，考虑到一般的云计算数据中心位于偏僻位置且集中管理，成本较低，而雾节点一般位于用户周围且分散管理，成本相对较高，因此我们设云服务提供商的报价 $c^{\mathrm{cloud}}=9\times10^{-10}$，雾服务提供商的报价 $c^{\mathrm{fog}}=10\times10^{-10}$，以确保付费和时延、能耗在同一数量级大小.另外，设雾服务提供商的折扣因子 $\alpha=0.04$.

其他方面，时延权重 λ_n^{T}、能耗权重 λ_n^{E} 及付费权重 λ_n^{C} 从[0.01, 1]中随机均匀选取，以尽可能模拟不同时延、能耗及付费权重的任务类型.以下仿真结果均为 1 000 次仿真结果的平均值.

5.2 系统平均代价

图 3 比较了 CATS 算法和 4 种基准方法在系统平均代价方面的表现：

1）本地计算（Local）.每个用户在本地设备处理其计算任务.

2）云计算（Cloud）.每个用户将其计算任务卸载到云上进行处理.

3）随机卸载（Random）.每个用户随机地决定是在本地处理其计算任务，还是将其计算任务卸载到云或某个雾节点上进行处理.

4）最优卸载（Optimal）.我们采用交叉熵[31]方法得到最小化系统整体代价的解.交叉熵是一种求解组和优化问题近似最优解的有效方法.具体的求解过程及参数设置可参考文献[32].

如图 3 所示，CATS 算法总能取得近似最优的系统平均代价，并且大大优于其他基准方法.这从系统整体角度验证了 CATS 算法的表现.另外，我们可以发现，通过引入雾层，可以大大降低用户的计算卸载代价.

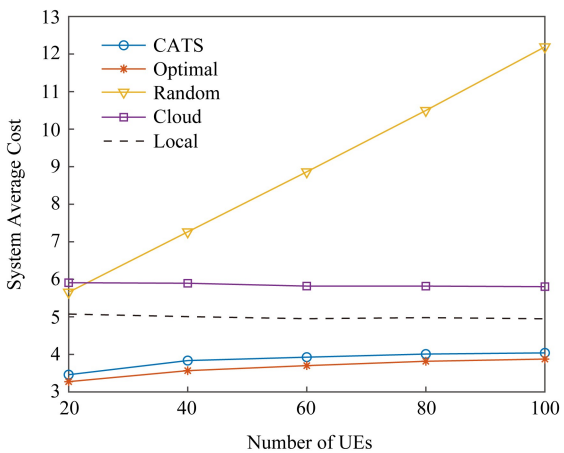

Fig. 3 System average cost vs number of UEs

图 3 系统平均代价 vs 用户数

5.3 受益用户数

图 4 比较了 CATS 算法和基准方法在受益用户数方面的表现.受益用户是指通过计算卸载,可以降低自身任务处理代价的用户(和本地计算相比),即$\{n\in\mathcal{N}|O_n<O_{n,l}\}$.这一指标表明了有多少用户可以通过计算卸载降低自身的任务处理代价.所以,受益用户数从个体方面反映了用户对任务调度或卸载结果的满意程度.

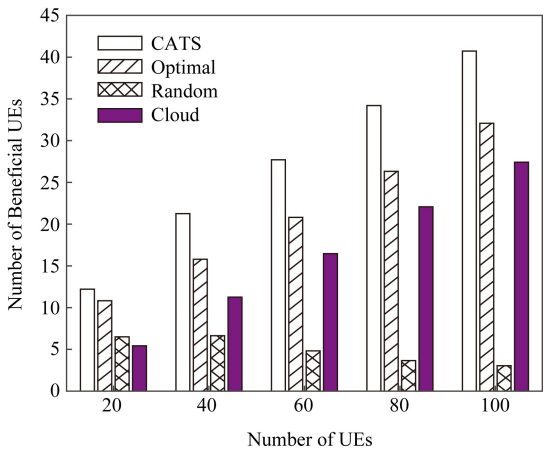

Fig. 4 Number of beneficial UEs vs number of UEs

图 4 受益用户数 vs 用户数

如图 4 所示,除了随机卸载之外,CATS 算法、最优卸载以及云计算的受益用户数都随着用户数的增加而增加,但 CATS 算法的受益用户数始终最大.这是因为以最小化系统代价为目标的最优卸载会损害用户个体利益,而通过将任务卸载建模为 CATS 博弈,可以协调用户之间的竞争、冲突,从而使得更多用户可以从中获益.所以,CATS 算法可以使得用户对调度结果更满意,因此结果也更稳定.这从用户个体角度验证了 CATS 算法的表现.

5.4 本地计算、雾计算及云计算用户分布

图 5 展示了 CATS 算法下,选择本地计算、雾计算及云计算的用户分布.如图 5 所示,随着用户数的增加,选择本地计算及云计算的用户也随之增加,这是因为雾节点的资源慢慢趋近饱和.

5.5 时延、能耗及付费成本分布

图 6 展示了 CATS 算法下,用户的平均时延、能耗及付费的成本分布.如图 6 所示,随着用户数的增加,时延及能耗成本也随之缓慢增加,逐渐占据主要成本.但付费成本反而随着用户数的增加而缓慢下降直至饱和.这是因为在动态付费模型下,单个用户的付费随着用户数的增加而降低,直至雾节点达到服务容量上限.

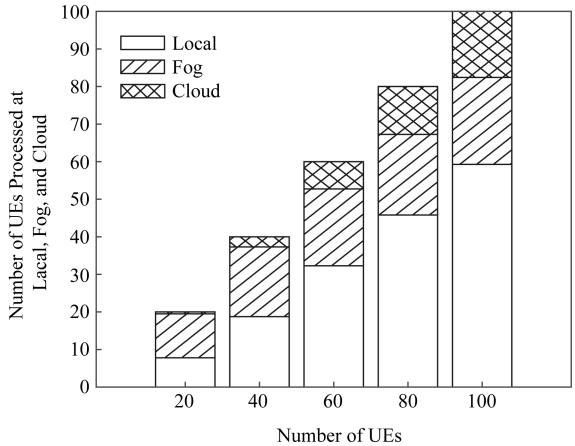

Fig. 5 Number of UEs processed at local, fog, and cloud vs number of UEs

图 5 本地计算、雾计算及云计算的用户数 vs 用户数

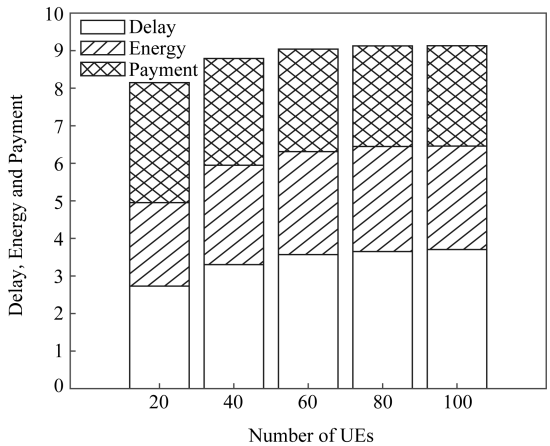

Fig. 6 Delay, energy and payment vs number of UEs

图 6 时延、能耗及付费 vs 用户数

5.6 收敛性

图 7 展示了 CATS 算法收敛所需的决策时隙数随用户数的变化关系.如图 7 所示,CATS 算法收敛所需的决策时隙数随着用户数的增长而线性增长,与我们推论 2 得到的结果基本吻合.这也表明,CATS 算法可以很好地适应网络规模,因此适用于大规模网络.

5.7 动态付费模型 vs 静态付费模型

图 8 比较了雾节点分别采用动态付费模型和静态付费模型的收入.所谓静态付费模型是指雾服务提供商对单位计算量收取的费用固定为 c^{fog},不受用户数影响,因此用户 n 向雾节点支付的费用为 $c^{\mathrm{fog}}z_n\gamma_n$.我们将动态付费模型的 CATS 算法和静态付费模型的 CATS 算法分别记作 CATS-dynamic 和 CATS-static.CATS 博弈在静态付费模型下对应

的博弈同样是一个广义势博弈，因此可以采用类似CATS算法的方法去寻找纯策略的纳什均衡解.由于内容限制，我们不再另做证明.

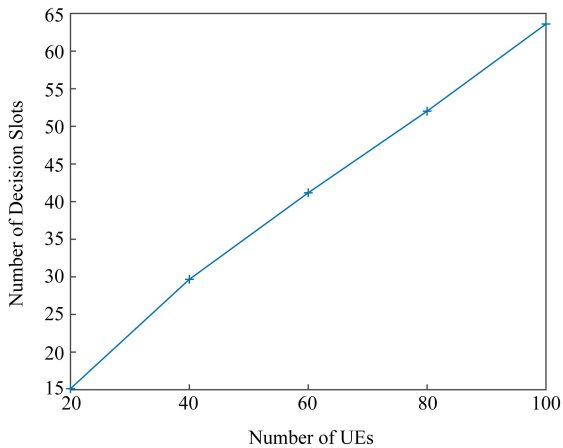

Fig. 7 Number of decision slots vs number of UEs

图7 决策时隙数 vs 用户数

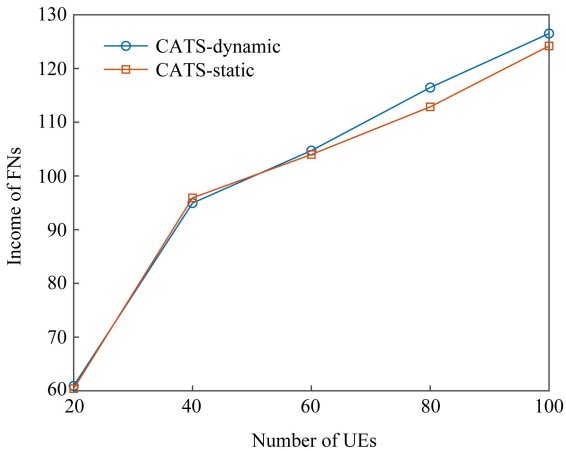

Fig. 8 Income of FNs vs number of UEs under different payment model

图8 不同付费模型下的雾节点的收入 vs 用户数

如图8所示，2种付费模型下，雾节点的收入都随着用户数的增加而增加.在用户数较小时，CATS-dynamic的雾节点收入和CATS-static的雾节点收入不相上下，但在用户数较大时，前者明显要大于后者.这是因为，在用户数较小时，雾节点无法吸引足够的用户，因此和云相比无法形成价格优势，从而使不同付费模型下雾节点服务的用户数可能一致.在服务用户数一致的情况下，静态付费模型显然可以帮助雾节点获得更多收入.但是，在用户数较大时，雾节点可以吸引足够的用户，动态模型这种“薄利多销”的模式可以帮助雾节点获得更多收入.因此，相比静态付费模型，动态付费模型可以让雾节点获得更多收入，尤其是在用户数较大时.

6 总 结

本文提出了一个云雾混合多层次算力网络及计算卸载系统，定义了一个由时延、能耗及付费组成的加权代价函数，并建模了一个代价感知任务调度CATS问题.而且，为激发云和雾的积极性，我们提出了一个基于计算量的付费模型并将付费相关代价也考虑进总代价.具体来说，根据云和雾的不同特性和需求，我们分别提出了一个静态付费模型和动态付费模型，从而构建了一个混合付费模型.为解决上述任务调度问题，我们提出了一个基于势博弈的问题分析框架.具体地，我们先将原问题构造为一个非合作博弈，然后通过势博弈理论证明了该博弈存在纯策略的纳什均衡解，最后利用有限提升性设计了一个分布式任务调度CATS算法去寻找纳什均衡解.我们最后进行了大量的数值仿真验证了算法在系统整体及用户个体2个维度上均具有优异的表现，即和集中式最优方法相比，CATS算法既可以取得近似最优的系统平均代价，又可以让更多用户从任务卸载中受益.此外，数值仿真结果也验证了，与静态付费模型相比，动态付费模型可以帮助雾在用户数较大时获得更多收入.

参 考 文 献

[1] Yang Yang. Multi-tier computing networks for intelligent IoT [J]. Nature Electronics, 2019, 2(1): 4-5

[2] Chiang M, Zhang Tao. Fog and IoT: An overview of research opportunities [J]. IEEE Internet of Things Journal, 2016, 3(6): 854-864

[3] Hu Yunchao, Patel M, Sabella D, et al. Mobile edge computing—A key technology towards 5G [R]. France: ETSI, 2015

[4] Jiang Mianheng. Urbanization and informatization: Historical opportunity for China's development [J]. Informatization Construction, 2010, 6: 8-9 (in Chinese)
(江绵恒. 城市化与信息化—中国发展的时代际遇[J]. 信息化建设, 2010, 6: 8-9)

[5] Kumar K, Lu Y H. Cloud computing for mobile users: Can offloading computation save energy? [J]. Computer, 2010, 43(4): 51-56

[6] Kumar K, Liu Jibang, Lu Y H, et al. A survey of computation offloading for mobile systems [J]. Mobile Networks and Applications, 2013, 18(1): 129-140

[7] Nowak D, Mahn T, Al-Shatri H, et al. A generalized Nash game for mobile edge computation offloading [C] //Proc of the 6th IEEE Int Conf on Mobile Cloud Computing, Services, and Engineering (Mobile Cloud). Piscataway, NJ: IEEE, 2018: 95-102

[8] Yang Yang, Liu Zening, Yang Xiumei, et al. POMT: Paired offloading of multiple tasks in heterogeneous fog networks [J]. IEEE Internet of Things Journal, 2019, 6(5): 8658-8669

[9] Pu Lingjun, Chen Xu, Xu Jingdong, et al. D2D fogging: An energy-efficient and incentive-aware task offloading framework via network-assisted D2D collaboration [J]. IEEE Journal on Selected Areas in Communications, 2016, 34(12): 3887-3901

[10] Chen Xu. Decentralized computation offloading game for mobile cloud computing [J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 26(4): 974-983

[11] Chen Xu, Jiao Lei, Li Wenzhong, et al. Efficient multi-user computation offloading for mobile-edge cloud computing [J]. IEEE/ACM Transactions on Networking, 2015, 24(5): 2795-2808

[12] Yang Lichao, Zhang Heli, Li Xi, et al. A distributed computation offloading strategy in small-cell networks integrated with mobile edge computing [J]. IEEE/ACM Transactions on Networking, 2018, 26(6): 2762-2773

[13] Li Ning, Martinez-Ortega J F, Rubio G. Distributed joint offloading decision and resource allocation for multi-user mobile edge computing: A game theory approach [EB/OL]. arXiv preprint, 2018 [2020-06-03]. https://arxiv.org/ftp/arxiv/papers/1805/1805.02182.pdf

[14] Souza V B C, Ramírez W, Masip-Bruin X, et al. Handling service allocation in combined fog-cloud scenarios [C] //Proc of ICC'16. Piscataway, NJ: IEEE, 2016: 1-5

[15] Liu Zening, Yang Xiumei, Yang Yang, et al. DATS: Dispersive stable task scheduling in heterogeneous fog networks [J]. IEEE Internet of Things Journal, 2018, 6(2): 3423-3436

[16] Deng Ruilong, Lu Rongxing, Lai Chengzhe, et al. Optimal workload allocation in fog-cloud computing toward balanced delay and power consumption [J]. IEEE Internet of Things Journal, 2016, 3(6): 1171-1181

[17] Chen Xu, Zhang Junshan. When D2D meets cloud: Hybrid mobile task offloadings in fog computing [C] //Proc of ICC'17. Piscataway, NJ: IEEE, 2017: 1-6

[18] Shah-Mansouri H, Wong V W S. Hierarchical fog-cloud computing for IoT systems: A computation offloading game [J]. IEEE Internet of Things Journal, 2018, 5(4): 3246-3257

[19] Yang Yang, Zhao Shuang, Zhang Wuxiong, et al. DEBTS: Delay energy balanced task scheduling in homogeneous fog networks [J]. IEEE Internet of Things Journal, 2018, 5(3): 2094-2106

[20] Nguyen D T, Le L B, Bhargava V. Price-based resource allocation for edge computing: A market equilibrium approach [J/OL]. IEEE Transactions on Cloud Computing, 2018 [2020-05-25]. https://ieeexplore.ieee.org/document/8373684

[21] Zhang Yanru, Han Zhu. Contract Theory for Wireless Networks [M]. Berlin: Springer, 2017: 73-88

[22] Liu Yang, Xu Changqiao, Zhan Yufeng, et al. Incentive mechanism for computation offloading using edge computing: A stackelberg game approach [J]. Computer Networks, 2017, 129: 399-409

[23] Zhang Huaqing, Zhang Yanru, Gu Yunan, et al. A hierarchical game framework for resource management in fog computing [J]. IEEE Communications Magazine, 2017, 55(8): 52-57

[24] Jošilo S, Dán G. A game theoretic analysis of selfish mobile computation offloading [C] //Proc of INFOCOM'17. Piscataway, NJ: IEEE, 2017: 1-9

[25] Jošilo S, Dán G. Selfish decentralized computation offloading for mobile cloud computing in dense wireless networks [J]. IEEE Transactions on Mobile Computing, 2018, 18(1): 207-220

[26] Liu Zening, Li Kai, Wu Liantao, et al. CATS_Appendix [EB/OL]. (2020-05-25) [2020-05-25]. https://mega.nz/file/TsgSFYBI#5ZNyMAjgxTSfaGDXUusA60W5fF4skVGybhJrTDDKpWY

[27] Monderer D, Shapley L S. Potential games [J]. Games and Economic Behavior, 1996, 14(1): 124-143

[28] Kwak J, Kim Y, Lee J, et al. DREAM: Dynamic resource and task allocation for energy minimization in mobile cloud systems [J]. IEEE Journal on Selected Areas in Communications, 2015, 33(12): 2510-2523

[29] Kim Y, Kwak J, Chong S. Dual-side optimization for cost-delay tradeoff in mobile edge computing [J]. IEEE Transactions on Vehicular Technology, 2017, 67(2): 1765-1781

[30] Mao Yuyi, Zhang Jun, Song S H, et al. Stochastic joint radio and computational resource management for multi-user mobile-edge computing systems [J]. IEEE Transactions on Wireless Communications, 2017, 16(9): 5994-6009

[31] Rubinstein R Y, Kroese D P. The Cross-entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning [M]. Berlin: Springer, 2013

[32] Liu Zening, Yang Yang, Zhou Mingtuo, et al. A unified cross-entropy based task scheduling algorithm for heterogeneous fog networks [C] //Proc of the 1st ACM Int Workshop on Smart Cities and Fog Computing (CitiFog'18). New York: ACM, 2018: 1-6

Liu Zening，born in 1993. Received his BSc degree in electrical science and technology from Wuhan University in 2015. PhD candidate with the School of Information Science and Technology，ShanghaiTech University. His main research interests include fog/edge computing，Internet of things，and artificial intelligence.

Li Kai，born in 1973. Received his BSc degree in signal and information processing from Tongji University in 2000. Senior engineer of ShanghaiTech University. His main research interests include the design and development of a new generation of wireless simulation platform and edge computing testbed.

Wu Liantao，born in 1989. Received his PhD degree in control science and engineering from Zhejiang University in 2017. He was also a visiting PhD student with the Department of Electrical and Computer Engineering，University of Wisconsin-Madison，Madison，USA. Currently research assistant professor with ShanghaiTech University. His main research interests include IoT，wireless communications，and compressive sensing.

Wang Zhi，born in 1969. Received his PhD degree from Shenyang Institute of Automation，Chinese Academy of Sciences，in 2000. Associate professor at Zhejiang University. His main research interests include acoustic signal and array signal processing，acoustic SLAM and robot positioning and navigation，indoor localization and temporal-spatial data mining，compressed sensing and deep learning，data fusion of multiple sensing systems，industrial Internet of things（IIoT），and privacy protection in crowdsourcing systems.

Yang Yang，born in 1974. Received his PhD degree in information engineering from the Chinese University of Hong Kong in 2002. Full professor at ShanghaiTech University. He is also an adjunct professor with the Research Center for Network Communication at Peng Cheng Laboratory，China. IEEE Fellow. His main research interests include fog computing networks，service-oriented collaborative intelligence，wireless sensor networks，IoT applications，and advanced testbeds and experiments.